Stratégies de recherche d'un intrus dans un entrepôt à l'aide d'un algorithme génétique

Pierre BODET (29301)

2024-2025

1 Introduction

La coopération de plusieurs robots pour rechercher et capturer un intrus mobile dans un entrepôt constitue un défi majeur en robotique autonome. Chaque robot peut choisir parmi un ensemble fini de stratégies de déplacement préalablement implémentées (par exemple : toujours tourner à droite, aller vers la dernière position où l'intrus a été vu, évoluer par rapport à son partenaire). L'enjeu est de définir, pour chaque robot, une distribution de probabilité sur ces stratégies afin de minimiser le temps moyen de capture d'un intrus. Ce dernier se déplaçant aléatoirement sans faire demi-tour et en fuyant un robot lorsqu'il en voit un.

Les algorithmes génétiques sont une méthode d'optimisation inspirée de la sélection naturelle, particulièrement adaptée à la recherche de stratégies efficaces malgré la présence d'aléatoire [1], ce qui rend les algorithmes génétiques utiles dans des structures labyrinthiques [2]. Le présent travail décrit un algorithme génétique conçu pour optimiser deux distributions de probabilité de dimension 30 (soit un total de 60 variables) en vue de la capture d'un intrus dans un entrepôt.

Ce rapport est structuré comme ceci : la Section 2 rappelle les principes généraux des algorithmes génétiques ; la Section 3 détaille la façon dont les différents points caractéristiques d'un algorithme génétique ont été implémentés dans notre travail. La Section 4 présente les résultats expérimentaux illustratifs. Enfin, la Section 5 conclut ce travail et et propose des pistes d'améliorations.

2 Principes généraux des algorithmes génétiques

Un algorithme génétique est une métaheuristique d'optimisation inspirée des mécanismes de l'évolution biologique. Les caractéristiques principales de ce type d'algorithme sont les suivantes. Ce type d'algorithme est composé d'une population, qui est un ensemble d'individus représentant une solution candidate. Chaque individu va être évalué à l'aide d'une fonction de score en simulant son comportement, pour décider de la façon dont se passera l'étape de croisement. Les individus vont mélanger leurs gènes pour créer des enfants. Pour éviter des convergences précoces dans des minima locaux, on ajoute une dernière étape de mutation qui va aléatoirement modifier des gènes des enfants. On recommence ensuite à l'étape d'évaluation avec comme nouvelle population les enfants ainsi générés.

Les algorithmes génétiques sont couramment utilisés dans des problèmes où l'espace de recherche est vaste, discontinu ou non différentiable, notamment en robotique et en optimisation combinatoire [3]. Dans notre cas, l'algorithme génétique permettra d'optimiser deux distributions de probabilité sur 30 stratégies chacune, en s'appuyant sur des simulations de capture pour évaluer la performance.

3 Spécifications de l'algorithme génétique appliqué

3.1 Représentation du génome

Chaque individu I est représenté par deux vecteurs réels :

$$I \;=\; \big(P^{(1)},\, P^{(2)}\big) \quad \text{où} \quad P^{(r)} = \big(p_{r,1},\, p_{r,2},\, \ldots,\, p_{r,30}\big)\,, \quad r \in \{1,2\}.$$

Les deux vecteurs vérifient les contraintes :

$$\sum_{i=1}^{30} p_{1,i} \;=\; 1, \quad \sum_{i=1}^{30} p_{2,i} \;=\; 1, \quad p_{r,i} > 0\,. \tag{1}$$

À chaque tour, le robot r sélectionne une stratégie $i \in \{1, \ldots, 30\}$ en tirant au sort selon la distribution $P^{(r)}$. Chaque stratégie correspond à un algorithme de déplacement prédéfini (tourner à droite, aller vers la dernière position où l'intrus a été vu, etc.).

3.2 Initialisation de la population

On choisit une population de taille N. Pour chaque individu initial, on génère deux vecteurs $\tilde{P}^{(1)},\ \tilde{P}^{(2)} \in \mathbb{R}^{30}_{>0}$ à valeurs uniformes sur $(0,1)$, puis on normalise :

$$P^{(r)} = \frac{\tilde{P}^{(r)}}{\sum_{i=1}^{30} \tilde{P}_i^{(r)}}, \quad r \in \{1,2\}.$$

Cela garantit une couverture aléatoire initiale de l'espace des distributions.

3.3 Evaluation du score

Pour évaluer un individu $I = (P^{(1)}, P^{(2)})$, on réalise 10 simulations distinctes :

1. On positionne l'intrus dans le coin inférieur droit.
2. Les deux robots démarrent depuis les deux coins à gauche de l'entrepôt.
3. L'intrus se déplace aléatoirement sans faire demi-tour et en fuyant un robot s'il le voit.
4. À chaque tour, chaque robot r tire une stratégie selon $P^{(r)}$, et applique le mouvement associé.
5. On incrémente le compteur de tours tant qu'aucune collision n'est détectée. Lorsque l'un des deux robots est sur une case adjacente à l'intrus, on enregistre le nombre de tours T_m.

Après 10 simulations, le score de l'individu I est

$$\text{Score}(I) \;=\; \frac{1}{10} \sum_{m=1}^{10} T_m. \tag{2}$$

L'algorithme cherche à minimiser cette valeur, c'est-à-dire à réduire le plus possible le temps moyen de capture.

3.4 Sélection et reproduction

À chaque génération t :

1. On évalue le score de tous les N individus $\{I_1^t, \ldots, I_N^t\}$.
2. On identifie l'individu élite $E = I_{\text{best}}^t$:

$$\text{Score}(E) = \min_{1 \le j \le N} \text{Score}(I_j^t).$$

3. Cet individu E se reproduit avec chacun des autres $N-1$ individus de la génération t. On obtient $N-1$ enfants, plus un clone de E, formant la nouvelle population $\{I_1^{t+1}, \ldots, I_N^{t+1}\}$.

Cette stratégie garantit une pression de sélection forte en faveur de l'élite, tout en permettant à chaque individu de contribuer à l'exploration de l'espace de solutions.

3.5 Opérateur de croisement uniforme par sous-vecteurs

Soient $E = (P_E^{(1)}, P_E^{(2)})$ l'élite et $P = (P_P^{(1)}, P_P^{(2)})$ l'autre parent. Le fils C est obtenu comme ceci :

1. Pour chaque indice de stratégie $i = 1, \ldots, 30$, on réalise une expérience de Bernoulli $b_i \sim \text{Bernoulli}(0{,}5)$.
   — Si $b_i = 1$, alors
   $$c_{1,i} = e_{1,i}, \quad c_{2,i} = e_{2,i}.$$
   — Sinon ($b_i = 0$),
   $$c_{1,i} = p_{1,i}, \quad c_{2,i} = p_{2,i}.$$
2. On obtient deux vecteurs non normalisés $\tilde{C}^{(1)} = (c_{1,1}, \ldots, c_{1,30})$ et $\tilde{C}^{(2)} = (c_{2,1}, \ldots, c_{2,30})$, que l'on renormalise :
   $$C^{(r)} = \left(\frac{c_{r,1}}{\sum_{k=1}^{30} c_{r,k}}, \ldots, \frac{c_{r,30}}{\sum_{k=1}^{30} c_{r,k}} \right), \quad r \in \{1, 2\}.$$
3. L'enfant $C = (C^{(1)}, C^{(2)})$ satisfait alors les contraintes de distribution.

Ce mode de croisement garantit la cohérence tactique : les probabilités associées à la même stratégie i sont transmises ensemble (pour le robot 1 et le robot 2) soit par l'élite, soit par le parent ordinaire, avec une probabilité de 50 % chacune.

3.6 Mutation

Après le croisement, chaque enfant subit, avec une probabilité $p_{\text{mut}} = 0{,}10$, un processus de mutation :
Pour chaque composante $i = 1, \ldots, 30$ du vecteur $C^{(1)}$, on réalise une expérience de Bernoulli $b_i \sim \text{Bernoulli}(p_{\text{mut}})$. Si $b_i = 1$, alors on applique ce qui suit :

1. On ajoute un bruit uniforme centré :
   $$c_{1,i} \leftarrow c_{1,i} + \varepsilon_{1,i}, \quad \varepsilon_{1,i} \sim \mathcal{U}(-0{,}1,\ 0{,}1),$$
   On procède de même pour $c_{2,i}$ avec $\varepsilon_{2,i} \sim \mathcal{U}(-0{,}1,\ 0{,}1)$.
2. Si l'une des nouvelles valeurs $c_{r,i}$ devient négative, on la remplace par un petit $\varepsilon_{\min} = 0{,}01$ pour assurer la présence de la stratégie.
3. On renormalise à nouveau chaque sous-vecteur :
   $$C^{(r)} = \frac{\tilde{C}^{(r)}}{\sum_{k=1}^{30} \tilde{c}_{r,k}}, \quad r \in \{1, 2\},$$
   où $\tilde{C}^{(r)}$ est le vecteur après ajout de bruit.

Cette mutation permet d'explorer localement l'espace des distributions et de maintenir une diversité suffisante pour éviter les minima locaux précoces.

3.7 Critère d'arrêt et paramètres

On fixe :
— Taille de population : 10 individus
— Nombre de simulations par évaluation : 10
— Nombre maximal de générations : 50
— Nombre de tours maximal d'une simulation : 10000

4 Résultats expérimentaux

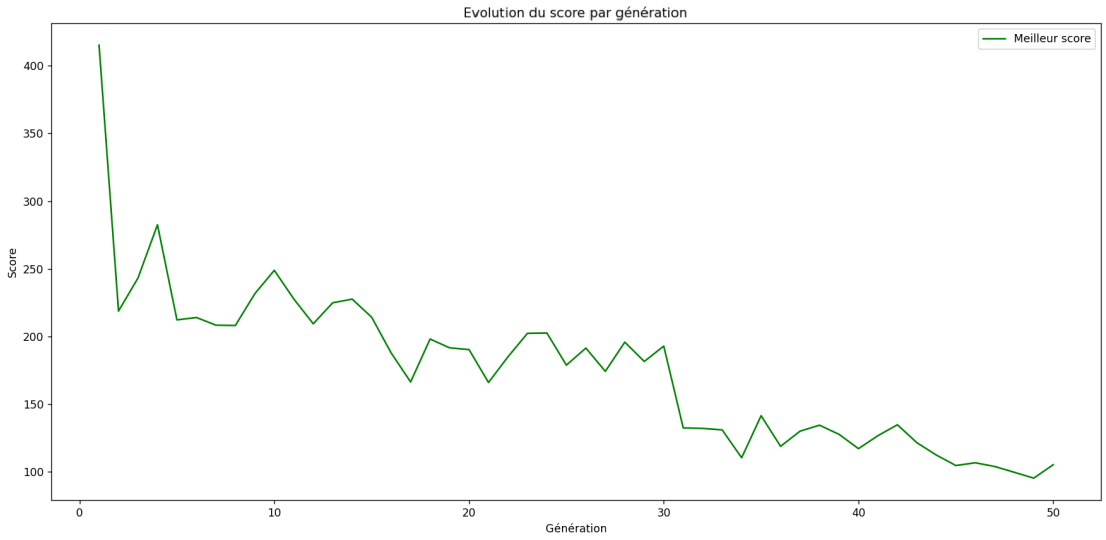
4.1 Création de l'entrepôt

On réalise un entrepôt de 25 × 25 cases en générant préalablement un labyrinthe, duquel des murs sont aléatoirement supprimés (selon une loi de Bernoulli $b_i \sim \text{Bernoulli}(0{,}5)$.

4.2 Évolution du score

La Table 1 présente, pour une simulation, l'évolution du score minimal (score de l'élite) sur certaines générations.

TABLE 1 – Évolution du score minimal au cours des générations



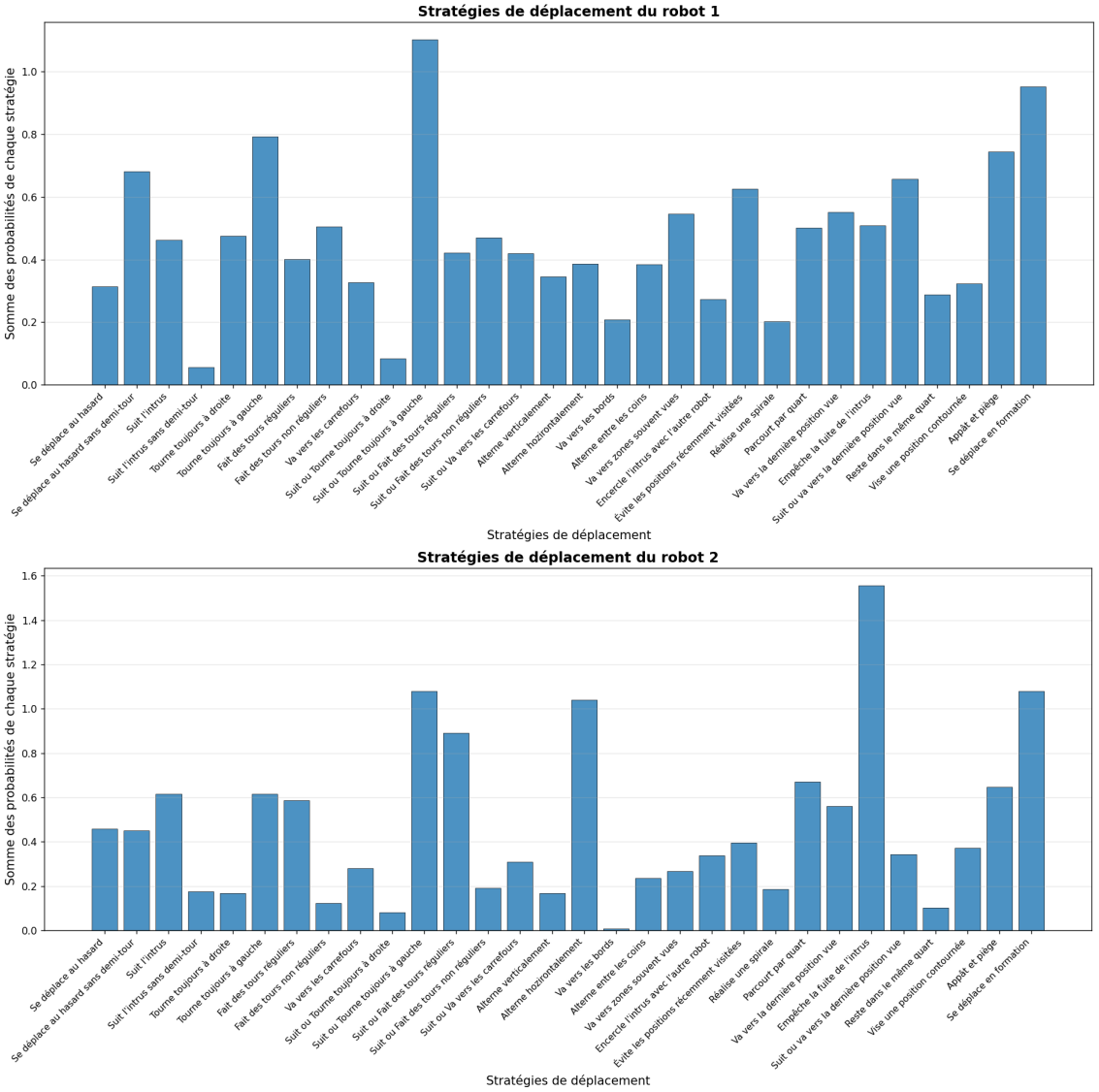
On observe que le score se réduit rapidement, puis stagne pendant un certain temps avec de décroître encore une fois. En effet, lors de la stagnation, les stratégies trouvées formaient un minimum local, et il a fallu diverses mutations pour réussir à en sortir et pouvoir améliorer le résultat. De plus, on voit des oscillations autour des valeurs de stagnation à cause de la probabilité de mutation qui est élevée, mais ce qui permet de rapidement trouver des solutions efficaces.

4.3 Distributions optimales

Si on somme les vecteurs optimaux $P_{\text{opt}}^{(1)}$ et $P_{\text{opt}}^{(2)}$ des individus optimaux lors de plusieurs utilisations du programme sur différents entrepôts, on obtient les résultats visibles dans la Table 2.

On remarque ainsi que les stratégies efficaces ne sont pas les mêmes pour les deux robots. En effet, la position initiale influence les stratégies qui fonctionneront.

TABLE 2 – Somme des vecteurs renvoyés par 10 lancements de l'algorithme génétique



5 Conclusion et pistes d'améliorations

5.1 Conclusion

Nous avons présenté un algorithme génétique qui permet d'optimiser la répartition probabiliste de 30 stratégies pour deux robots coopérant dans un entrepôt. Chaque individu est défini par deux vecteurs de 30 probabilités, contraints à somme unitaire. L'opérateur de croisement uniforme par sous-vecteurs garantit la transmission conjointe des probabilités associées à la même stratégie pour les deux robots, et la mutation (10 %) préserve la diversité. Les expérimentations montrent une convergence en moins de 50 générations, réduisant fortement le temps moyen de capture.

5.2 Pistes d'améliorations

Pour améliorer les résultats obtenus et éviter les oscillations générées par notre programme, nous pourrions réduire la probabilité de mutation tout en augmentant le nombre de générations. L'algorithme mettrait alors plus de temps à s'exécuter, mais les résultats seraient beaucoup plus stables. Il est également possible d'envisager l'utilisation du principe de cooling, c'est-à-dire de réduire progressivement la probabilité de mutation au fil des générations. Cela permet d'obtenir une exploration importante des stratégies lors des premières générations, tout en favorisant, au fil du temps, le fait que les stratégies trouvées soient mieux explorées en réduisant les oscillations. [4].

Augmenter la taille de la population pourrait aussi nous permettre d'augmenter la diversité des solutions potentielles testées, mais cela augmentera considérablement le temps de calcul alors que les améliorations obtenues pourraient rester limitées.

Nous pourrions enfin revoir les stratégies développées, effacer celles qui ne sont jamais sélectionnées et ajouter plus de variantes à celles qui sont le plus souvent choisies pour essayer d'optimiser davantage les résultats.

Références

[1] Mitchell, M. (1998). *An Introduction to Genetic Algorithms.* MIT Press.

[2] Choubey, N. (2012). *A-Mazer with Genetic Algorithm.* Researchgate.

[3] Khaled, G. (2007). *Optimisation combinatoire par métaheuristiques.* Technip.

[4] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning.* Addison-Wesley.

Filières MP/MPI - ENS de Paris-Saclay, Lyon, Rennes et Paris - Session 2025

Page de garde du rapport de TIPE

Attention: Le rapport doit être réalisé par l'étudiant(e).
Si le rapport résulte d'une collaboration, elle doit être clairement annoncée.

NOM : BODET	Prénoms : Pierre Hervé Jérôme
Classe : MPI*	
Lycée : Faidherbe	Numéro de candidat : 29301
Ville : Lille	

Concours auxquels vous êtes admissible (les indiquer par une croix) :

Banque MP

ENS Paris-Saclay	MP - Option MP	
	MP - Option MI	
ENS de Lyon	Math - Option M-MP	
	Math - Option M-MI	
	Informatique	
ENS Rennes	MP - Option MP	
	MP - Option MI	
ENS Paris	MP - Option P	
	MP - Option I	

Banque MPI

ENS Paris-Saclay	Option Math	
	Option Info	X
ENS de Lyon	Math MPI	
	Info MPI	X
ENS Rennes	Info MPI	X
ENS Paris	Option Math	
	Option Info	

Matière dominante du TIPE (la sélectionner d'une croix inscrite dans la case correspondante) :

Informatique	X	Mathématiques		Physique	

Titre du TIPE : Stratégies de recherche d'un intrus dans un entrepôt à l'aide d'un algorithme génétique

Nombre de pages (à indiquer dans les cases ci-dessous) :

Texte	5	Illustration	2	Bibliographie	1

Attention, les illustrations doivent figurer dans le corps du texte et non en fin du document !

Résumé ou descriptif succinct du TIPE (6 lignes, maximum) :

Dans ce TIPE, nous réalisons une simulation de recherche d'un intrus par deux robots dans un entrepôt labyrinthique. Nous implémentons 30 stratégies de déplacements différentes pour les robots et nous utilisons un algorithme génétique afin de trouver les combinaisons de stratégies les plus efficaces.

À Lille

Le 5 juin 2025

Signature du (de la) candidat(e)

Pierre

Signature du professeur responsable de la classe préparatoire dans la discipline

Cachet de l'établissement

Lycée FAIDHERBE
9, rue Armand Carrel
B.P. 767 - 59034 LILLE Cedex
Tél. : 03.20.60.5000
Fax : 03.20.60.5005

La signature du professeur responsable et le tampon de l'établissement ne sont pas indispensables pour les candidats libres (hors CPGE).